Software Seguro

```
root@software-seguro:~# ./start_course.sh
```

Clase 5: Security Misconfiguration

Curso Profesional de Ciberseguridad Web y Ethical Hacking

Repaso labs Clase 4

Objetivos de la Sesión

Identificar y explotar configuraciones débiles, omisiones en el endurecimiento del servidor y fuga involuntaria de datos internos.

Security Misconfiguration

Es una categoría de vulnerabilidad que ocurre cuando los sistemas, servicios o aplicaciones no están configurados de manera segura por defecto, o cuando sus configuraciones no se mantienen actualizadas y alineadas con las mejores prácticas de seguridad.

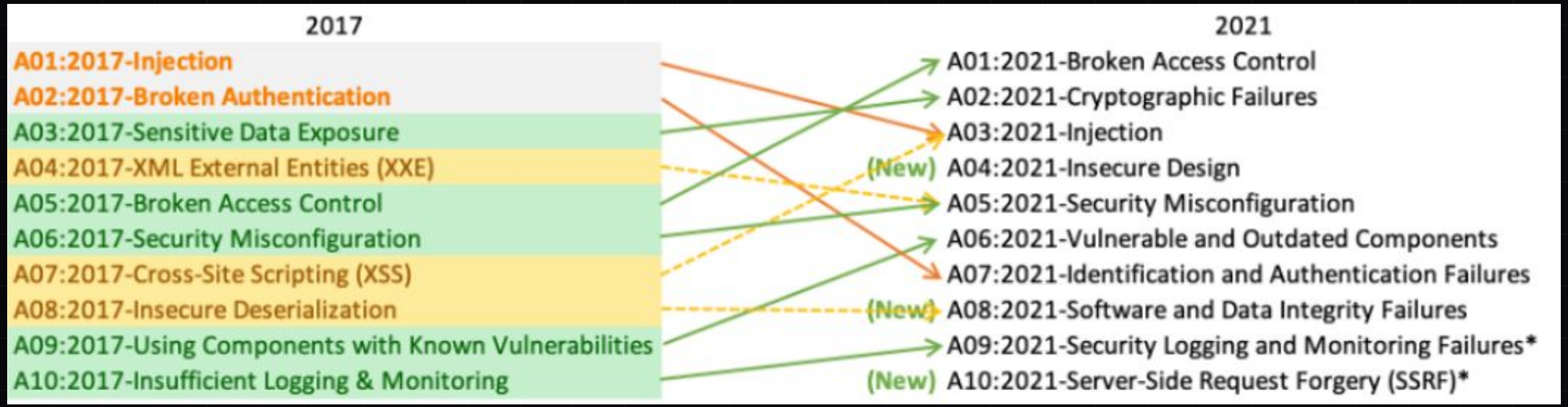
La Puerta Trasera Lógica

Ocurre cuando la arquitectura, los servicios o el software están implementados con valores predeterminados inseguros, incompletos o mal endurecidos.

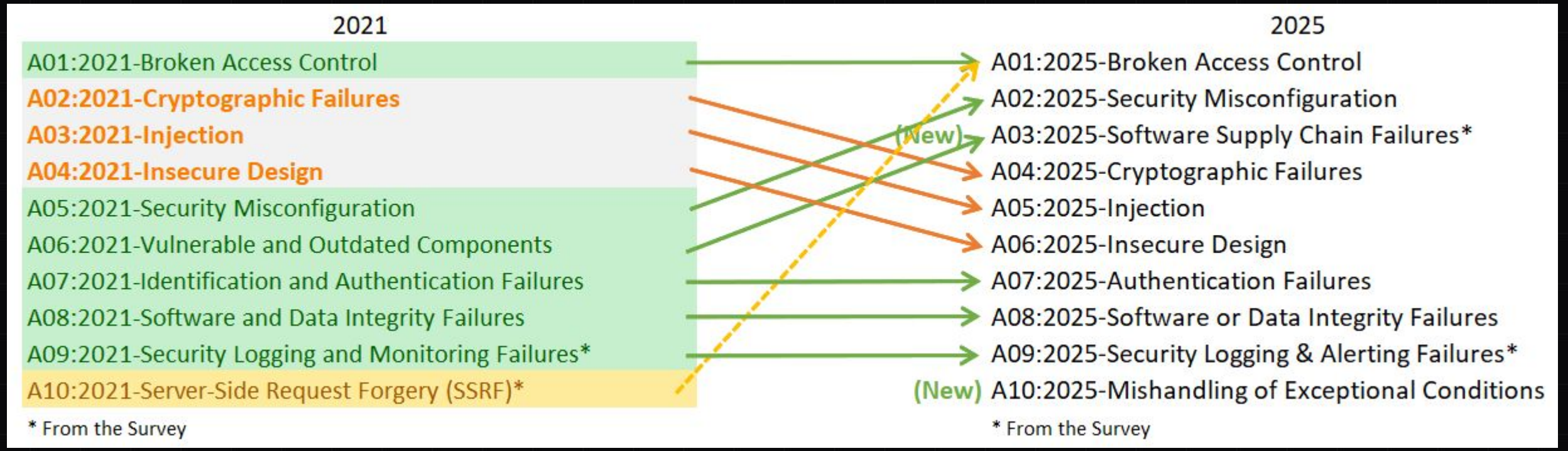
El Vector Más Frecuente

A diferencia de fallas lógicas complejas, la mala configuración suele derivar de omisiones humanas en despliegues rápidos a producción.

Ubicación según OWASP



Ubicación según OWASP



Cabeceras de seguridad

Las cabeceras de seguridad son instrucciones especiales que el servidor web envía al navegador a través de la respuesta HTTP para controlar el comportamiento del navegador y así proteger al usuario y a la aplicación frente a distintos tipos de ataques.

Cabecera	Previene...	¿Cómo lo hace?
Content-Security-Policy	**XSS, inyección de scripts, ataques DOM**	Restringe de dónde se pueden cargar scripts, imágenes, estilos, etc. Por ejemplo: bloquea scripts inline o externos no autorizados.
X-Content-Type-Options: nosniff	**MIME sniffing → ejecución no deseada**	Obliga al navegador a respetar el tipo de contenido declarado (Content-Type), evitando que ejecute archivos como scripts si no deberían serlo.
Strict-Transport-Security	**MITM, downgrade attacks, SSL stripping**	Obliga al navegador a usar siempre HTTPS, incluso si el usuario escribe http:// en la URL. Evita conexiones inseguras.
X-Frame-Options	**Clickjacking**	Evita que tu sitio sea embebido en un <iframe> en otro dominio. Protege al usuario de hacer clic en botones invisibles superpuestos.
Referrer-Policy	**Filtración de URLs sensibles**	Controla si se envía o no el encabezado Referer. Evita que otros sitios vean rutas internas, tokens en la URL, etc.
Permissions-Policy	**Abuso de APIs del navegador**	Restringe el acceso a funciones sensibles como cámara, ubicación, micrófono, etc., desde sitios no autorizados.

Configuración de Cookies & Entropía

Sin flags de seguridad y Sin suficientes bits de entropía

HttpOnly

Impide el acceso a document.cookie vía JavaScript, bloqueando el robo de tokens por XSS.

Secure

Garantiza envío exclusivo por HTTPS

SameSite

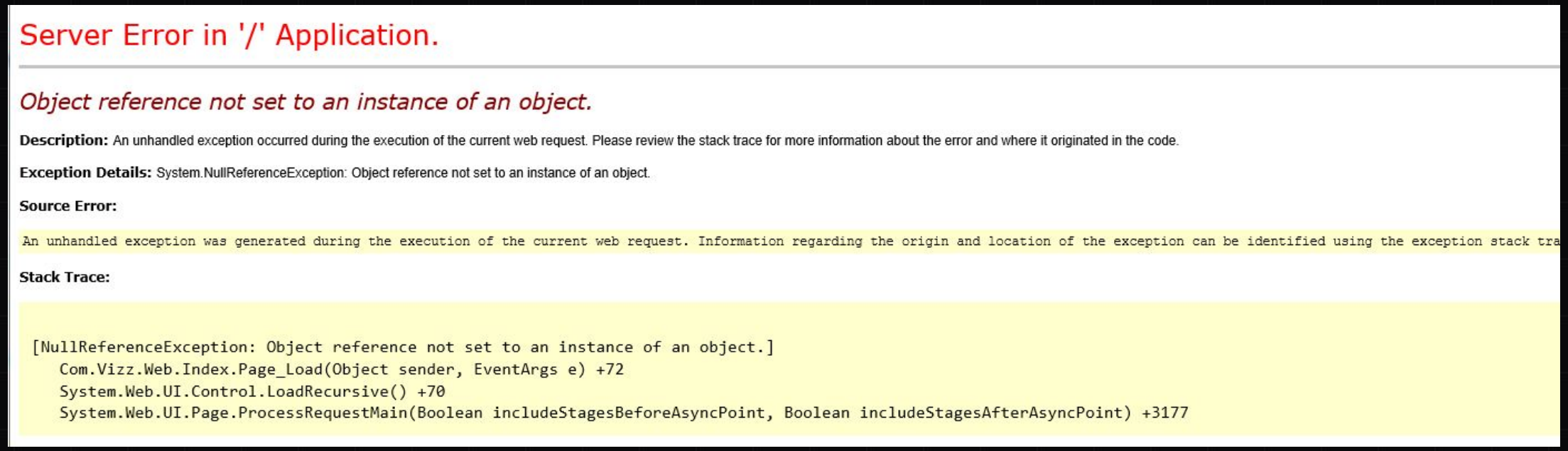
previene ataques de falsificación de peticiones entre sitios (CSRF). Strict; Lax (ojo); None

Baja Entropía

IDs predecibles o basados en contadores permiten ataques de fuerza bruta y secuestro de cuentas.

Manejo de errores

Cuando una aplicación muestra mensajes de error con información técnica como trazas, rutas internas o versiones del sistema, expone detalles que un atacante puede usar para planear ataques más efectivos. La mejor práctica es mostrar mensajes genéricos al usuario y guardar los detalles técnicos únicamente en los registros del servidor.



Manejo de errores

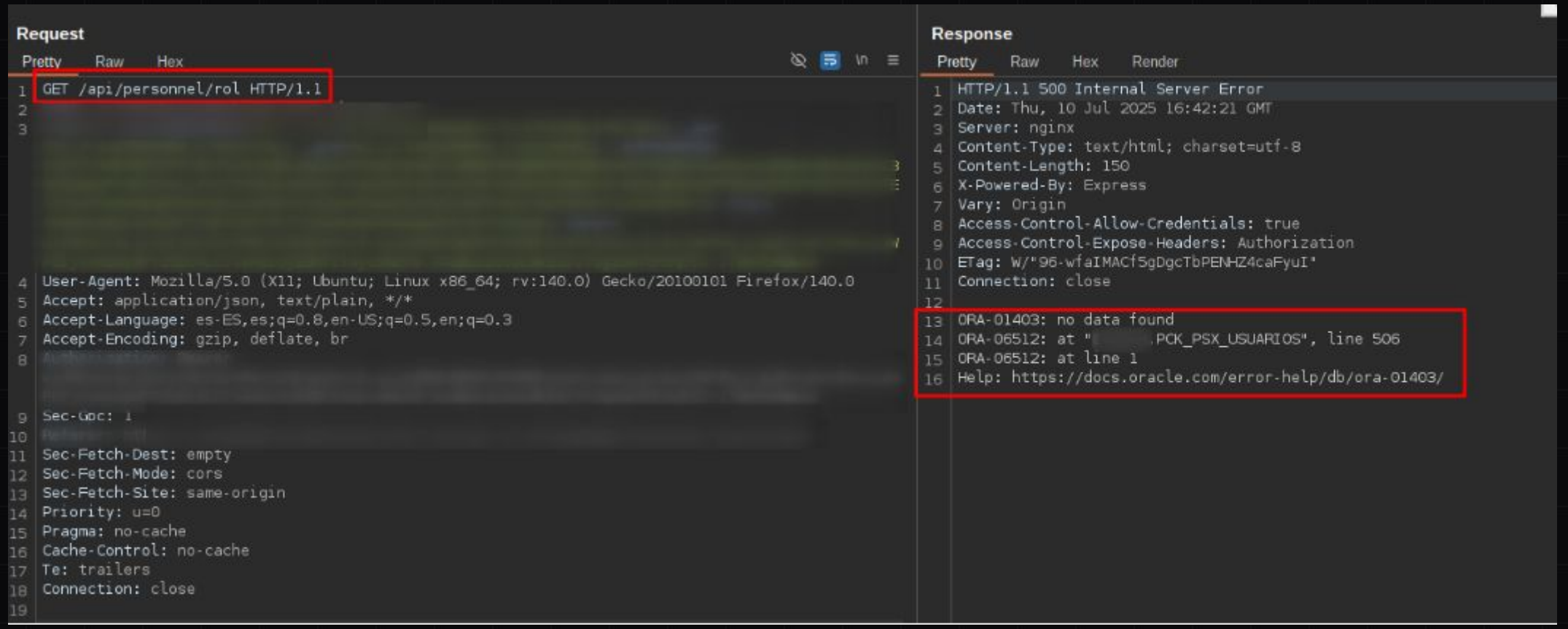
La idea: entregarle al servidor algo que su código no previó, para que una excepción no capturada escape a la respuesta.

Agrupamos los disparadores por vector de entrada.

- Comillas y metacaracteres en cualquier parámetro: ' " ` \ ; --. Disparan errores de SQL, ORM o parsing.
- Tipo equivocado: texto donde va un entero (id=abc), negativos (id=-1), o números enormes que desbordan (id=999..).
- Array donde va un escalar: ?id[]=1 o ?id[]=1&id[]=2.
- JSON malformado o con tipos cambiados: cuerpo truncado {"id":, o {"id":{}} / {"id":[1,2]} donde se espera un número.
- Campos faltantes o de más: quitar un campo obligatorio, o inyectar claves que el deserializador no espera.
- Payloads enormes (cuerpo de varios MB) → límites de memoria o de parseo.
- Demasiados parámetros en una request → algunos frameworks lanzan error de configuración (y revelan el límite).
- Romper la deserialización de sesiones, cookies firmadas o tokens (cambiar un byte del valor).
- Content-Type que no corresponde: XML a un endpoint JSON o JSON con Content-Type: application/x-www-form-urlencoded
- Caracteres de control y Unicode raro: null byte %00, saltos de línea %0a%0d, overlong UTF-8, emojis.
- Valores gigantes: un header o parámetro de cientos de KB puede pegar contra límites internos.
- Verbos HTTP raros: PUT, DELETE, PATCH, TRACE o uno inventado (FOO /) en endpoints que sólo manejan GET/POST.

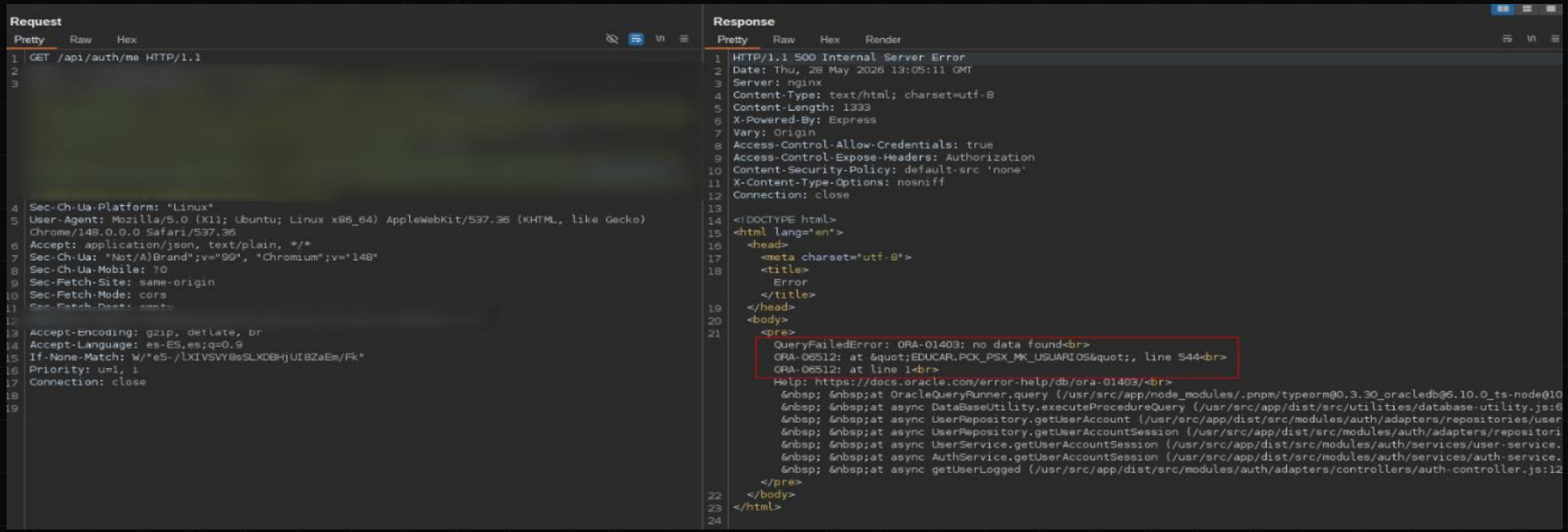
Ejemplos reales

- Navegar a la funcionalidad que permite agregar roles o permisos a un usuario.
- Interceptar la petición original y cambiar el método HTTP de PATCH a GET.
- Reenviar la solicitud modificada.

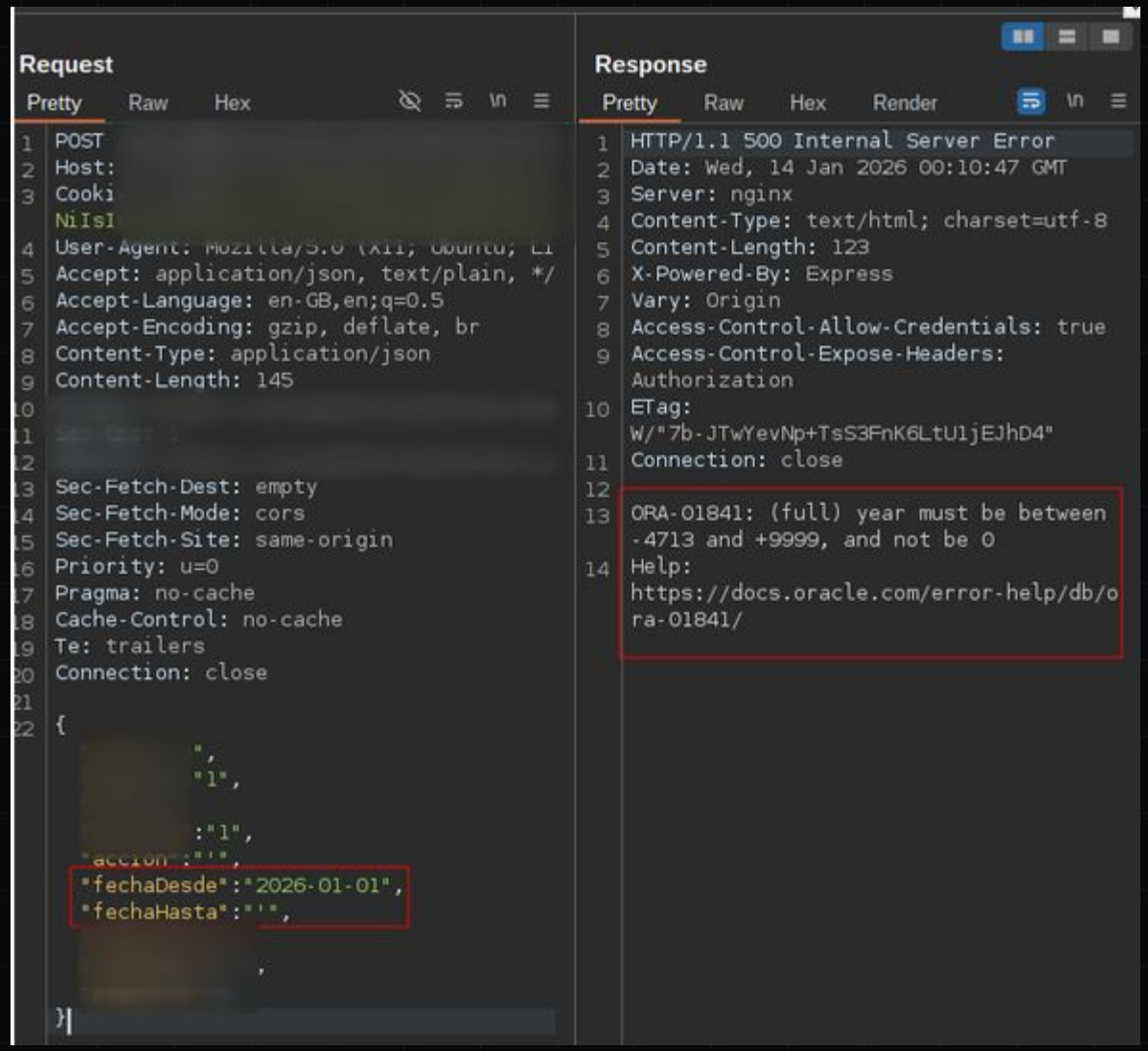


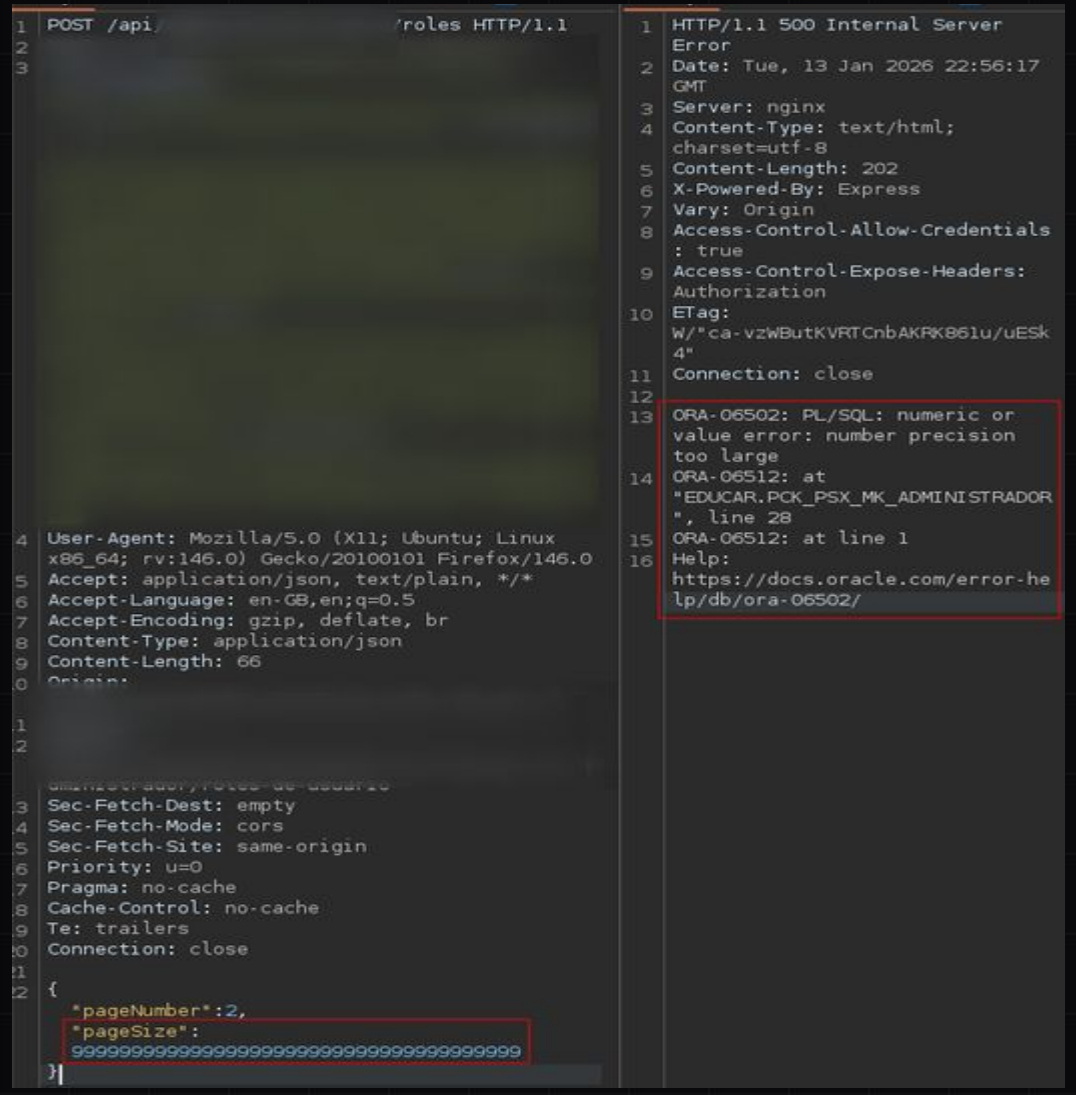
Ejemplos reales

- Se setea en cookie un token temporal hasta la selección de rol, (copiar ese token)
- Seleccionar rol aspirante,
- Se setea en cookie otro token que reemplaza al anterior
- Intercambiar ese token con el token copiado en el paso dos

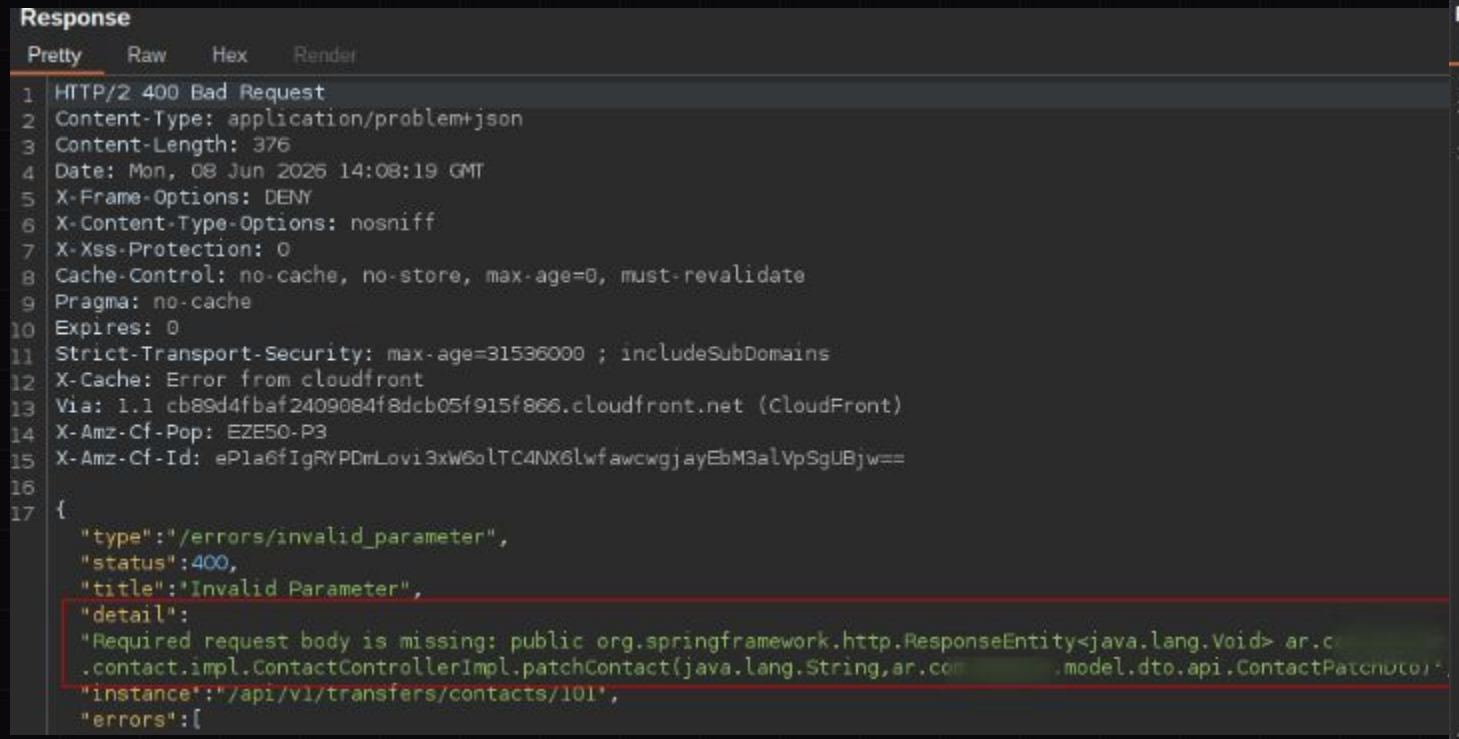


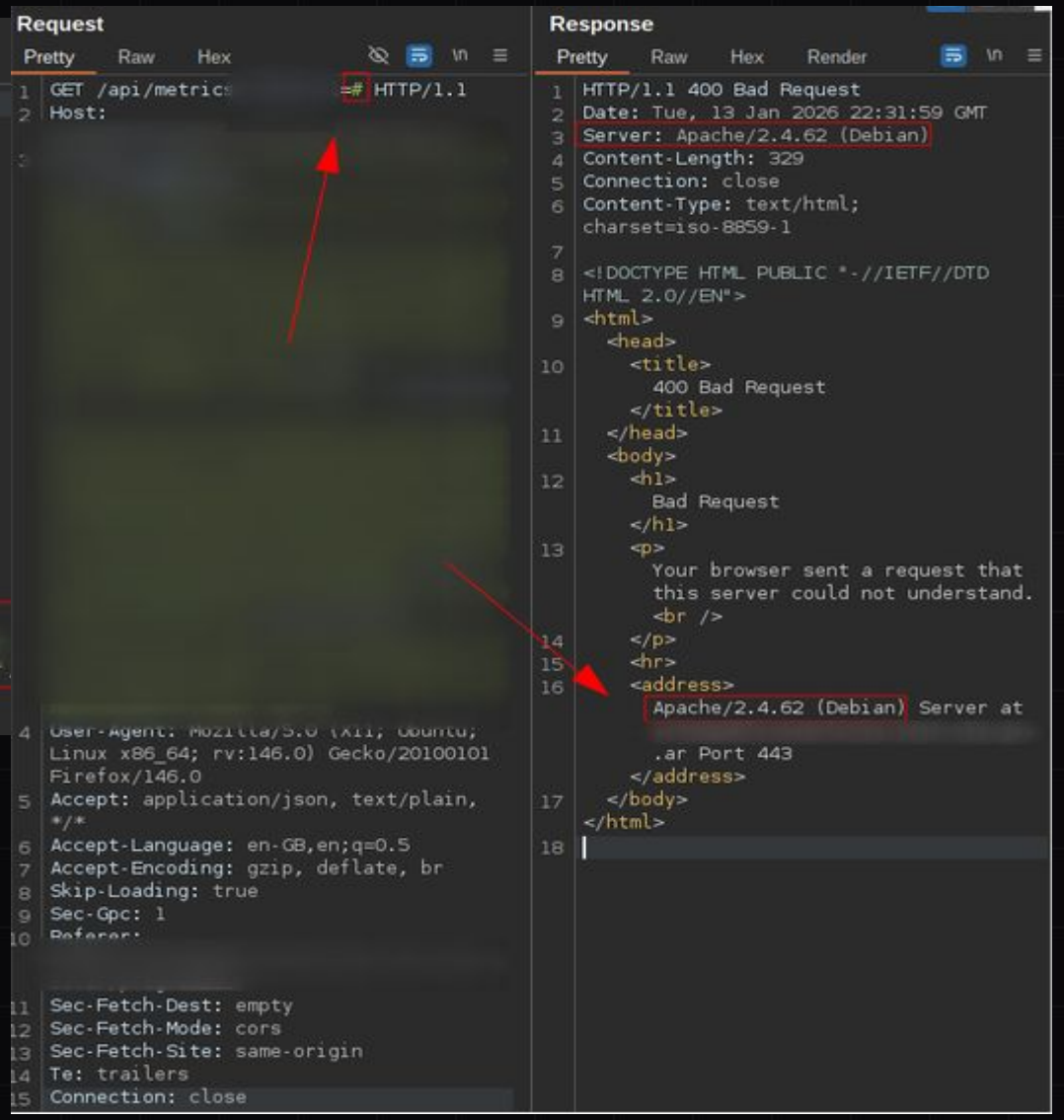
Ejemplos reales



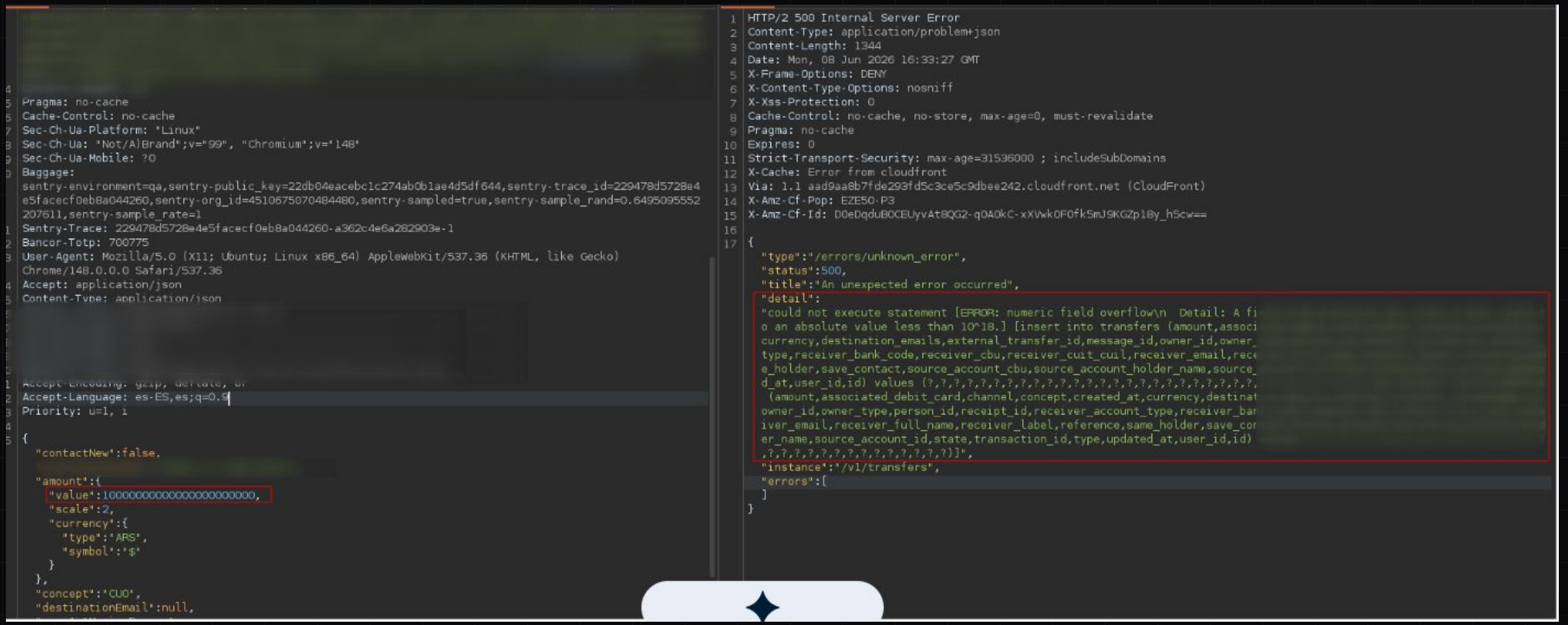


Ejemplos reales





Ejemplos reales



```
Pragma: no-cache
Cache-Control: no-cache
Sec-Ch-Ua-Platform: "Linux"
Sec-Ch-Ua: "Not/A)Brand";v="99", "Chromium";v="148"
Sec-Ch-Ua-Mobile: ?0
Baggage:
sentry-environment=qa,sentry-public_key=22db04eacebc1c274ab0b1ae4d5df644,sentry-trace_id=229478d5728e4
e5facecf0eb8a044260,sentry-org_id=4510675070484480,sentry-sampled=true,sentry-sample_rand=0.6495095552
207611,sentry-sample_rate=1
Sentry-Trace: 229478d5728e4e5facecf0eb8a044260-a362c4e6a282903e-1
Bancor-Totp: 700775
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
Chrome/148.0.0.0 Safari/537.36
Accept: application/json
Content-Type: application/json
Accept-Encoding: gzip, deflate, br
Accept-Language: es-ES,es;q=0.9
Priority: u=1, i

{
  "contactNew":false,

  "amount":{
    "value":10000000000000000000000,
    "scale":2,
    "currency":{
      "type":"ARS",
      "symbol":"$"
    }
  },
  "concept":"CUO",
  "destinationEmail":null,
```

```
HTTP/2 500 Internal Server Error
Content-Type: application/problem+json
Content-Length: 1344
Date: Mon, 08 Jun 2026 16:33:27 GMT
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-Xss-Protection: 0
Cache-Control: no-cache, no-store, max-age=0, must-revalidate
Pragma: no-cache
Expires: 0
Strict-Transport-Security: max-age=31536000 ; includeSubDomains
X-Cache: Error from cloudfront
Via: 1.1 aad9aa8b7fde293fd5c3ce5c9dbee242.cloudfront.net (CloudFront)
X-Amz-Cf-Pop: EZE50-P3
X-Amz-Cf-Id: D0eDqduBOCEUyvAt8QG2-qOA0kC-xXVwkOFOfk5mJ9KGZp18y_h5cw==

{
  "type":"/errors/unknown_error",
  "status":500,
  "title":"An unexpected error occurred",
  "detail":
  "could not execute statement [ERROR: numeric field overflow\n  Detail: A fi
  o an absolute value less than 10^18.] [insert into transfers (amount,associ
  currency,destination_emails,external_transfer_id,message_id,owner_id,owner_
  type,receiver_bank_code,receiver_cbu,receiver_cuit_cuil,receiver_email,rece
  e_holder,save_contact,source_account_cbu,source_account_holder_name,source_
  d_at,user_id,id) values (?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,?,
   (amount,associated_debit_card,channel,concept,created_at,currency,destinat
  owner_id,owner_type,person_id,receipt_id,receiver_account_type,receiver_bar
  iver_email,receiver_full_name,receiver_label,reference,same_holder,save_cor
  er_name,source_account_id,state,transaction_id,type,updated_at,user_id,id)
  ,?,?,?,?,?,?,?,?,?,?,?,?,?,?)]",
  "instance":"/v1/transfers",
  "errors":[
  ]
}
```

HTTP Parameter Pollution

HPP consiste en enviar el mismo parámetro más de una vez. El estándar HTTP no define qué debe pasar, así que cada stack elige distinto: uno toma el primero, otro el último, otro los concatena, otro arma un array. El ataque vive en esa discrepancia: si un componente valida un valor y otro usa otro, se abre un hueco.

Matriz de parsing por tecnología Comportamiento aproximado para color=red&color=blue:

Tecnología	Resultado	Valor
ASP.NET / IIS	Concatena con coma	`red,blue`
ASP clásico	Concatena con coma	`red,blue`
PHP / Apache	Último	`blue`
JSP / Tomcat (`getParameter`)	Primero	`red`
Python Flask / Werkzeug (`args.get`)	Primero	`red`
Python Django	Último	`blue`
Node.js / Express	Array	`['red','blue']`
Perl / CGI	Array / primero	`red`
Ruby on Rails	Último	`blue`
Go `net/http` (`FormValue`)	Primero	`red`

Path Traversal y Fallas de Codificación

Surge cuando la aplicación utiliza parámetros de usuario directamente para abrir archivos del sistema sin sanitizar rutas relativas.

La app arma una ruta de archivo con input del usuario sin canonicalizar ni validar. Insertando secuencias ../ (o sus variantes codificadas) salís del directorio permitido y leés —a veces escribís— archivos arbitrarios del sistema.

Fingerprint: cualquier parámetro que huela a ruta o archivo: ?file=, ?page=, ?doc=, ?template=, ?path=, ?download=, ?img=, ?lang=, ?include=.

Payloads y bypasses

```
# Básico (Linux / Windows)
?file=../../../../etc/passwd
?file=..\..\..\..\windows\win.ini

# Si filtran la cadena ../ literal:
..%2f..%2f..%2fetc/passwd          # URL-encoding
..%252f..%252fetc/passwd           # doble encoding
..%c0%af..%c0%afetc/passwd         # overlong UTF-8
../../../../etc/passwd%00.png      # null byte (apps viejas)
....//....//etc/passwd             # replace no recursivo de ../
```

CORS (Cross-Origin Resource Sharing)

Por defecto, los navegadores web aplican una regla de seguridad estricta llamada Same-Origin Policy (SOP) (Política del Mismo Origen).

¿Qué es un "Origen"? Está compuesto por tres elementos: el esquema (ej. https), el dominio (ej. mi-sitio.com) y el puerto (ej. 443). Si cambia cualquiera de estos tres, se considera un origen diferente.

Cuando un script en el navegador intenta hacer una petición de origen cruzado (ej. un fetch desde malicioso.com hacia api.banco.com), ocurren dos cosas según el tipo de petición:

Peticiones Simples

: Métodos básicos (GET, POST, HEAD) con cabeceras estándar. El navegador sí envía la petición al servidor y adjunta la cabecera Origin: https://malicioso.com. Luego, el navegador mira la respuesta del servidor; si el servidor no incluye las cabeceras CORS permitidas, el navegador bloquea la lectura de la respuesta para el script malicioso.

Peticiones Preflight (Verificación previa)

Ocurren con métodos complejos (PUT, DELETE) o cuando se envían cabeceras personalizadas (como Content-Type: application/json). El navegador envía primero una petición de tipo OPTIONS preguntando: "¿Me permitís hablar con vos?". Si el servidor responde autorizando al origen, recién ahí el navegador dispara la petición real.

Las Cabeceras Clave de CORS

El intercambio se basa en el diálogo entre el cliente y el servidor mediante estas cabeceras:

Access-Control-Allow-Origin (ACAO): Indica qué dominios tienen permiso para leer la respuesta. Puede ser un dominio específico (https://misitio.com), un comodín (*) para cualquier dominio, o ser reflejado dinámicamente.

Access-Control-Allow-Credentials (ACAC): Un booleano (true o false). Indica si el navegador permite enviar credenciales (como cookies de sesión o tokens de autenticación) junto con la petición de origen cruzado.

Access-Control-Allow-Origin: * junto a Access-Control-Allow-Credentials: true.
El resultado: Los estándares de los navegadores modernos bloquean esto explícitamente porque es un riesgo absurdo, pero si una API legacy o mal implementada lo acepta o se combina con malas validaciones de proxy, expone los datos de sesión de cualquier usuario.

Variables de entorno, backups y configs

Qué buscamos: .env, dumps y copias que el editor o el deploy dejaron servidos como archivos estáticos.

Qué contienen: Credenciales de base de datos, tokens de pasarelas de pago y el secreto de firma JWT en texto plano.

Cómo llegamos: Fuzzing de rutas con diccionarios de backups/configs (ffuf, feroxbuster, dirsearch).

El hallazgo: Un directorio .git/ accesible expone objetos, refs e índice — todo el repositorio, no sólo la web.

Reconstrucción: git-dumper baja los objetos y rearma el árbol completo en local para leerlo con git normal.

Análisis histórico: gitleaks/truffleHog recorren TODO el historial: una credencial “borrada” sigue viva en un commit viejo.

El Error Común

Dejar carpetas `.git`, archivos `.env`, respaldos `.bak` o dumps SQL directamente accesibles en la raíz pública del servidor web.

Almacenamiento Correcto

Usar gestores dedicados (Vault, AWS Secrets Manager), excluir archivos sensibles en servidores web y restringir permisos de lectura.

Otras Cabeceras de Seguridad Esenciales

Cabecera	Previene...	¿Cómo lo hace?
`Content-Security-Policy`	XSS, inyección de scripts	Restringe el origen de carga de scripts y recursos.
`X-Content-Type-Options`	MIME sniffing	Obliga a respetar el Content-Type declarado.
`Strict-Transport-Security`	MITM, downgrade attacks	Fuerza el uso exclusivo de HTTPS.
`X-Frame-Options`	Clickjacking	Evita que el sitio sea embebido en un iframe.
`Referrer-Policy`	Filtración de URLs	Controla la información enviada en el encabezado Referer.

Demo

Asignación para el Hogar

Resolver y enviar los objetivos del Módulo II: Reporte y Brechas: Security Misconfiguration

Ingresar a la plataforma de Software Seguro, en las opciones les aparecera el curso “Curso Profesional de Ciberseguridad Web y Ethical Hacking”, dentro del curso tienen los distintos módulos con sus objetivos que deberán enviar antes de la próxima sesión

¿Preguntas?

Fin de la Sesión 5.

https://softwareseguro.com.ar